Friend on Campus

Service containerization report | CS3219 | 20 October 2026

FoC is packaged as independently built application images, coordinated by Docker Compose. The local stack contains four Go services, the implemented Next.js frontend, an Nginx gateway, four service-owned PostgreSQL instances and Mailpit. Order also serves its own embedded web UI at **/orders**. [R1-R3]

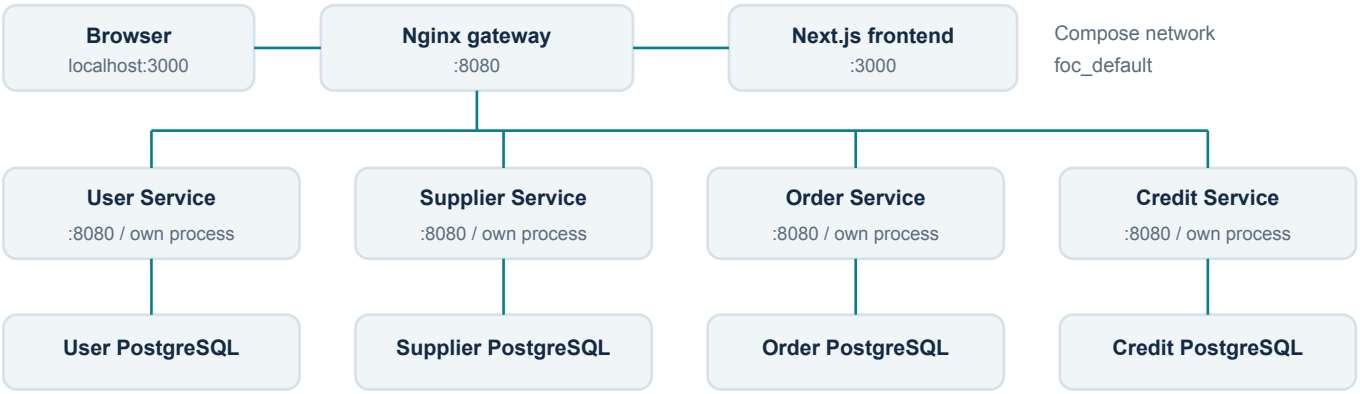


Figure 1. Public requests enter the gateway. Each backend owns a separate database. Cross-service HTTP and User-to-Mailpit SMTP are additional internal links described on page 5.

Container / component	Container listener	Host access / responsibility
gateway	8080	127.0.0.1:3000; frontend and public API routing
frontend	3000	Gateway only; Next.js account/catalogue/wallet UI
user-service	8080 public; 8081 internal	127.0.0.1:8080; accounts, sessions and roles
supplier-service	8080	127.0.0.1:8083; catalogue and internal read routes
order-service	8080	127.0.0.1:8084; errand lifecycle, UI and events
credit-service	8080 public; 8081 internal	127.0.0.1:8082; wallets, escrow and history
user-db / supplier-db / order-db / credit-db	5432 each	No host ports; four distinct databases and volumes
mailpit	1025 SMTP; 8025 web	127.0.0.1:8025 inbox; local verification emails

Scope of this report. Implementation details are checked against the current working tree, based on commit **dc4a3bd**. Static configuration validation passed. A live build/start demonstration could not be executed because the configured Docker daemon socket was absent; page 7 provides the exact procedure and evidence to collect.

The main runtime graph has 11 containers. Test databases, Go check containers, frontend checks, smoke and browser runners are opt-in profiles rather than production dependencies.

Dockerfiles and independent delivery

Go services: one reusable build pattern

Each backend has its own Dockerfile and build context. The **development** stage uses **golang:1.27.1-bookworm**, sets **/app**, copies **go.mod** and **go.sum**, runs **go mod download**, then copies the source. Copying dependency manifests first lets ordinary source edits reuse the dependency layer. [R2]

```
FROM development AS build
RUN CGO_ENABLED=0 go build -trimpath -o /user-service ./cmd/server

FROM debian:bookworm-slim AS runtime
# Install CA certificates; User also installs curl for its healthcheck.
COPY --from=build /user-service /usr/local/bin/user-service
USER 10001:10001
ENTRYPOINT ["user-service"]
```

Supplier, Order and Credit use the same pattern with their respective binary names. The final image contains the compiled executable and runtime OS packages, without the Go compiler or module cache. SQL migrations are embedded in each binary; Order embeds its UI assets too. The executable starts directly through the JSON-form ENTRYPOINT and handles SIGTERM with up to 15 seconds of HTTP shutdown. [R2, R5]

Frontend, gateway and test targets

The frontend uses **node:22-bookworm-slim**. Its development stage runs **npm ci** from the committed lockfile; the build stage runs **npm run build**. Next.js standalone output is copied into a fresh runtime stage with static assets. It runs as **node**, with **NODE_ENV=production**, binding 0.0.0.0:3000 and starting with **node server.js**. [R2, R4]

The gateway extends **nginxinc/nginx-unprivileged:1.28-alpine** only to copy its routing configuration. The inherited image startup runs Nginx on port 8080. Go tests target the development stage; frontend checks use its development stage. Order UI tests use the separate **mcr.microsoft.com/playwright:v1.58.2-noble** target, so browsers and Node test tools do not enter the Order runtime image. [R1-R3]

Why services can evolve independently

Each service owns its source folder, Go module, image, configuration, database and migrations. Internal HTTP contracts connect them without shared database access. A service change can be rebuilt and recreated alone, for example **docker compose up -d --build --no-deps order-service** once its dependencies are running. Service check profiles use disposable databases and HTTP fakes; Credit and Order also provide service-local Compose files. Standalone Order can explicitly join the root dependency network using its integration overlay. [R1, R6]

> **Nice-to-haves.** The project brief selects proximity sorting, dynamic pricing, reviews and cloud deployment. Separate images and service-owned schemas let these be added to the owning service or a new component without rebuilding every backend. The same images can support the planned EC2/Compose deployment with environment-specific settings and persistent storage. These features and cloud provisioning are backlog items, not demonstrated deployments. [R8]

This follows Docker guidance on multi-stage builds, dependency-layer reuse, small runtime images and decoupled components [E1]. Application containers run non-root, read-only, with all capabilities dropped and no-new-privileges; frontend/gateway receive writable /tmp tmpfs mounts. This hardening is not configured identically for the pre-built database and Mailpit containers. [R1-R2]

Reproducible configuration and secrets

Dependency and version management

Layer	Committed reproducibility mechanism
Go backends	Go 1.27.0 module directives; Go 1.27.1 build image; go.mod selects dependency versions and go.sum verifies module content. All four use pgx v5.11.0; User also uses x/crypto v0.57.0.
Frontend	Node 22 image; package-lock.json plus npm ci. package.json fixes Next.js 16.3.6 and React/React DOM 19.3.0; ranged development dependencies resolve through the lockfile.
Runtime and infrastructure	Debian bookworm-slim; PostgreSQL 18-bookworm; Mailpit v1.29; unprivileged Nginx 1.28-alpine; Playwright 1.58.2-noble for Order UI tests.
Shared workflow	Compose files, per-service Dockerfiles and scripts are committed. CI invokes the same container-based service and frontend checks as developers.

Reproducibility limit: image tags remain mutable and apt packages are not version-pinned. The setup controls application dependencies and major/minor platform choices, but does not promise byte-identical rebuilds. Recording immutable image digests and deploying registry images tagged by commit would strengthen release reproducibility; those are improvements, not current configuration. [R1-R2, R4, R7; E1]

Configuration enters at runtime

The root Compose file includes Credit's service-local Compose file and Order's local file plus a root integration overlay. Include-level **env_file** entries supply interpolation values from the root and service .env files. User additionally receives its root .env through the container-level **env_file**; other services receive explicit **environment** mappings. Root overrides align Order's origin and User/Credit/Supplier tokens. [R1]

```
environment:                        # excerpt: supplier-service
  USER_SERVICE_INTERNAL_URL: http://user-service:8081
  USER_INTERNAL_TOKEN: ${USER_INTERNAL_TOKEN:?Initialize User first}
  USER_APP_ORIGIN: ${USER_APP_ORIGIN:-http://localhost:3000}
```

The Go configuration loaders validate required values, credentials and URL/origin settings before serving requests. Missing configuration or failed migrations stops startup. The public frontend value **NEXT_PUBLIC_USER_API_URL** is a build argument and must contain only non-secret browser configuration; **USER_SERVICE_URL** is supplied to the frontend at runtime. [R1, R4-R5]

Keeping credentials out of source, images and logs

User, Credit and Order initializers generate random local credentials, preserve existing settings and create new secret files with restrictive permissions. The root .gitignore excludes .env files; service .dockerignore files exclude generated .env files from build contexts. Real credentials are injected at container creation, not passed as secret build arguments or copied into runtime images. Committed .env.example files use placeholders. [R2, R4]

Application logs use generic configuration/database failure messages and sanitized dependency errors rather than credential-bearing URLs or request bodies. Gateway access logging is disabled to avoid logging query data. Use **docker compose config --quiet** for validation: plain **config** prints resolved secrets. Environment injection still exposes secrets to authorized Docker operators; an external secret manager or mounted Compose secrets would be a production improvement. Mailpit intentionally displays verification codes in its local inbox. [R3-R5]

Database lifecycle and persistent data

Initialization and schema evolution

Each service uses its own pre-built **postgres:18-bookworm** container, with its own POSTGRES_USER, POSTGRES_DB and randomly generated password. The official image initializes an empty data directory on first startup. Four named volumes mount at **/var/lib/postgresql**, the volume location used by this PostgreSQL 18 image family. No custom database Dockerfile is required. [R1; E2]

Before opening HTTP listeners, each Go executable applies its own numbered SQL files embedded with **go:embed**. The migration runner sorts filenames, opens a transaction, takes a PostgreSQL advisory lock, checks **schema_migrations** and applies only missing migrations. Recording migration names and the schema changes in the same transaction prevents partial committed initialization. Future schema changes add numbered files; applied migrations are not edited. [R5]

For Order's workflow upgrade, stop all old Order instances first. The new binary performs resumable legacy bootstrap after migration and before workers/listeners start. Original deadlines and existing financial jobs survive. Mixed old/new writers are unsupported, so independent deployment does not imply a rolling upgrade is safe for every schema change. [R5-R6]

Seed data and business initialization

Supplier migration **002_seed.sql** imports 21 supplied supplier records and their campus locations once; later catalogue migration adds fields. Migration tracking prevents restarts from duplicating seed rows or undoing administrator edits. User has no default account or admin password. The initial admin must register, verify and claim the designated one-time role. Credit's 100-credit allocation is triggered by new-account verification through an idempotent private API, not by a startup seed script. [R5-R6]

Operation / storage	What happens to application data?
stop / start or restart	Named PostgreSQL volumes remain attached. Accounts, sessions, suppliers, orders, wallets, history and durable retry work survive. Uncommitted transactions roll back on process failure.
Rebuild / recreate containers	Replacing application or database containers reuses the same named volumes within the Compose project. New binaries apply only unapplied migrations.
docker compose down	Containers and the project network are removed; named volumes remain. Starting the same project reattaches its databases.
docker compose down -v	Removes project-managed named volumes and destroys the local application databases. This is a deliberate reset, not a normal stop command.
Test storage and tmpfs	Dedicated test PostgreSQL containers use tmpfs rather than application volumes. Data is disposable; frontend/gateway /tmp is also ephemeral.
Mailpit inbox	No persistent volume is configured. Captured verification emails are development state and are not preserved as durable application data.

Operational implications. Changing POSTGRES_PASSWORD in .env does not rotate an existing database role password; initialization settings act on an empty database. Root and standalone Compose project names also produce different volume identities, so existing standalone Order data is not migrated into the root stack automatically. Volumes provide persistence on the Docker host, not backups; automated backups or managed database recovery are not configured. [R1, R6; E2]

Service discovery and communication isolation

How components locate one another

The root stack uses Compose's default network, normally **foc_default**. Docker DNS resolves service names, so connections use container ports rather than host mappings. For example, User connects to **user-db:5432** and sends local SMTP to **mailpit:1025**; a container's localhost refers to itself. Nginx uses Docker's DNS resolver **127.0.0.11** with a 10-second validity interval to refresh upstream addresses after recreation. [R1, R3]

Traffic	Destination / contract	Protection
Browser to application	localhost:3000 -> gateway:8080 -> frontend:3000 or a public service :8080	One browser origin; public routes are explicitly mapped.
Protected API to User	user-service:8081 /internal/v1/sessions/validate	Bearer USER_INTERNAL_TOKEN plus the session token; validate each request and fail closed.
User to Credit	credit-service:8081 /internal/v1/wallets/{id}/initial-allocation	Separate Credit bearer credential; idempotent allocation.
Order to Credit	credit-service:8081 /internal/v1/escrows/{orderId}, /release and /payout	Credit bearer credential; stable order IDs and idempotent financial operations.
Order to Supplier	supplier-service:8080 public catalogue for user actions; /internal/v1/... for background reads	User session on foreground reads; separate SUPPLIER_INTERNAL_TOKEN for internal reads.
Service to database	Own database DNS name on port 5432	Distinct database credentials; no cross-service table access.

External and internal access are separated in layers

Listener and routing boundary. User and Credit publish only public port 8080, keeping their separate port 8081 off the host. No database port is published. The gateway rejects every **/internal** path with 404, rejects unknown /api paths, and routes only the implemented public APIs and pages. EXPOSE in a Dockerfile documents a listener; Compose **ports** determines host publication. [R1-R3]

Authentication boundary. Private routes require backend bearer credentials. Public protected routes resolve identity through User instead of trusting a browser-supplied user ID or caching authorization. Browser mutations enforce the configured exact Origin, with local **foc_session** cookies. Internal service credentials are not delivered to browser JavaScript. [R4-R6]

> **Exact isolation achieved.** This is a local stack with host publication, gateway filtering and authenticated internal APIs. It does not define separate edge and database networks or a network marked internal:true: containers share one network. Direct loopback development ports remain available, and Supplier's internal routes share its published listener, so they rely on the separate bearer credential outside the gateway. A production setup should remove debug publications, segment networks and terminate HTTPS; these changes are not already implemented.

Order's standalone integration overlay explicitly joins the existing foc_default network while retaining its own default network for its database. Merely starting a separate Compose project does not make the root service DNS names reachable. [R1, R6]

Startup readiness and recovery after outages

Compose startup dependencies

Each backend waits for its database with **condition: service_healthy**. PostgreSQL runs **pg_isready** every 3 seconds, with a 3-second timeout and 20 retries. User also depends on Credit/Mailpit; Supplier on User; Order on User/Supplier/Credit; frontend on User; and gateway on frontend/Supplier/Order. Starting gateway transitively starts the full graph. [R1]

Sibling dependencies generally use **service_started**, which confirms process startup rather than API readiness. Each Go service has a 30-second migration context and exits on migration failure; there is no indefinite database-start retry loop. After building, the startup script waits up to 120 seconds for running containers and configured healthchecks. [R1, R5; E3]

Failure / timing	Implemented behavior
Database not ready at startup	Compose holds back the dependent backend until the database healthcheck passes. If initialization/migrations still fail or time out, the application exits with a sanitized error; fix the cause and rerun startup.
HTTP sibling starts late or is unavailable	The app may already be listening. Bounded HTTP requests return a dependency error; authentication fails closed rather than accepting stale identity. Clients retry after recovery.
Database unavailable during runtime	Each backend /readyz returns 503 when Ping fails, while /healthz still reports process liveness. Database operations fail safely; pgx can reconnect on later work once the database returns. A failed request is not automatically replayed.
Credit unavailable during verification	User commits verification and a durable allocation job. Verified users can log in while their wallet is pending. The worker retries with backoff, capped at 300 seconds, and resumes after restart.
Credit unavailable during Order settlement	Refund/payout intent and jobs remain in PostgreSQL. Workers retry stable identities after recovery or lost responses; idempotent Credit endpoints prevent duplicate financial effects.
Mailpit / SMTP unavailable	User registration can return 503 after saving a pending account. The client retains registration state and uses resend after recovery; no automatic SMTP delivery retry worker is implemented.

Durable asynchronous workflow without a broker

Order commits lifecycle events in PostgreSQL. Independent in-process consumers project timers, settlement jobs and browser publications, committing effects and receipts atomically. They poll in bounded one-second passes; consumer backoff is capped at 60 seconds. A separate recovery loop defaults to 30 seconds for creation cleanup, financial retries and Supplier reconciliation. The durable feed supports authenticated SSE and reconnect replay; Nginx disables stream buffering. [R3, R5-R6]

No RabbitMQ or Redis is configured, so there is no broker startup dependency. PostgreSQL persists this workflow. Supplier outages do not imply deletion, and restarts retain original deadlines. [R1, R6]

> **Readiness has a defined limit.** Only User currently has a Compose-level application healthcheck. All four backends expose /readyz, but these checks measure database reachability rather than downstream availability or empty queues. No restart policy is declared, and Compose does not automatically restart dependents when another container becomes unhealthy. Explicit probes and the smoke journey are needed to verify the whole system.

Reproducible startup and demonstration

Run from the repository root with Docker Engine/Desktop running, a compatible Compose plugin supporting **include** and **up --wait**, a POSIX shell and OpenSSL. No host Go, Node or PostgreSQL installation is needed. The canonical command is **sh scripts/start.sh**. It performs the following steps in order. [R1, R4, R7]

```
sh user-service/scripts/init-env.sh
sh credit-service/scripts/init-env.sh
sh order-service/scripts/init-env.sh
docker compose config --quiet
docker compose up --build -d --wait --wait-timeout 120 gateway
```

1. Confirm startup and database readiness

```
docker compose ps
curl --fail http://localhost:3000/healthz
curl --fail http://localhost:8080/readyz
curl --fail http://localhost:8082/readyz
curl --fail http://localhost:8083/readyz
curl --fail http://localhost:8084/readyz
```

Collect build completion output, the running/healthy container listing and HTTP 200 readiness responses. Gateway /healthz checks only Nginx; the four direct probes exercise each backend's database readiness. If startup fails, inspect recent logs with **docker compose logs --tail=100** followed by the affected service names. Do not include resolved environments or secret values in submission screenshots.

2. Demonstrate a real user journey

Open **http://localhost:3000**; register a unique school-domain account, retrieve its verification code at **http://localhost:8025**, verify and log in. Check the seeded supplier catalogue and initial wallet. With a second verified account, create an errand at **/orders**, accept/deliver it and confirm receipt; inspect both wallets and transaction history. These checks cover routing, authentication, database initialization and the financial integration. [R6-R7]

On a fresh User database, configure and claim the designated first admin before the smoke script: fixture promotion consumes the one-time bootstrap claim. Then run **sh scripts/smoke.sh**. The script creates and cleans temporary fixtures; CI additionally exercises funded Order lifecycle behavior. [R7]

3. Verify independent containerized checks

```
docker compose --profile test run --build --rm user-tests
docker compose --profile test run --build --rm supplier-tests
docker compose --profile test run --build --rm credit-tests
docker compose --profile test run --build --rm order-tests
docker compose --profile test run --build --rm frontend-checks
```

Backend checks enforce formatting, vet, race-enabled tests and compilation. Frontend checks run ESLint, TypeScript and a production build. To demonstrate persistence, create a record, run **docker compose down**, rerun **sh scripts/start.sh** and verify it remains. Do not use **-v** for this exercise. [R6-R7]

> **Evidence recorded while preparing this report (20 October 2026).**
> PASS: Docker Compose v5.5.1 is installed; **docker compose config --quiet** exited 0 using the existing local environment; shell syntax validation of startup and all three initializers exited 0.
> NOT VERIFIED: image builds, running containers, endpoint probes, smoke and service test execution. Docker could not connect because the configured desktop-linux socket **~/.docker/run/docker.sock** did not exist. The commands above are a reproducible demonstration procedure, not a claim of successful runtime execution.

REFERENCES AND IMPLEMENTATION EVIDENCE

Sources and image acknowledgements

Repository references identify the files used to substantiate this report. Paths are relative to the FoC repository. Current configuration and code take precedence over older backlog/status descriptions. No cloud resources, broker deployment or successful runtime test results are inferred from documentation alone.

Reference	Repository evidence
R1 - Compose topology	compose.yaml; credit-service/compose.yaml; order-service/compose.yaml; order-service/compose.root.yaml; order-service/compose.integration.yaml. Ports, dependencies, volumes, environment mappings, test profiles and runtime restrictions.
R2 - Image builds	Dockerfile in user-service/, supplier-service/, order-service/, credit-service/, frontend/ and gateway/; service .dockerignore files. Build stages, users, startup commands and copied runtime assets.
R3 - Gateway	gateway/default.conf. Public route allowlist, /internal rejection, Docker DNS resolver, authentication rate limiting, log settings and SSE buffering configuration.
R4 - Dependencies / settings	Each backend go.mod and go.sum; frontend/package.json, package-lock.json and next.config.ts; .gitignore and .env.example; user-service/scripts/init-env.sh, credit-service/scripts/init-env.sh and order-service/scripts/init-env.sh.
R5 - Runtime / storage	Each service cmd/server/main.go and internal/{service}/config.go, http.go and store.go; embedded migrations/*.sql; user-service/internal/user/credits.go; order-service/internal/order/dependencies.go.
R6 - Service contracts	README.md and API.md in each backend folder; frontend/README.md. Ownership, authentication, retry, idempotency, bootstrap and standalone integration behavior.
R7 - Run / verification	README.md; scripts/start.sh; scripts/smoke.sh; service scripts/check.sh; .github/workflows/ci.yml. Reproducible startup and configured checks; not proof of a completed CI run.
R8 - Nice-to-have scope	docs/project-brief.md, Selected nice-to-haves and Deployment strategy. Proximity sorting, dynamic pricing, reviews and EC2/cloud updates are selected backlog scope.

Pre-built image sources

PostgreSQL is the **Docker Official Image**, used directly as **postgres:18-bookworm** [E2]. Mailpit is the upstream **Axllent/Mailpit** image, used directly as **axllent/mailpit:v1.29** [E4]. Neither uses a custom application Dockerfile. No messaging-broker image is deployed. The gateway derives from the upstream **NGINX unprivileged image** [E5].

Application build/runtime bases come from Docker Official Images: golang, debian and node. Order UI testing uses Microsoft's Playwright Docker image. The source choices are acknowledgements of upstream images, not a claim that their current tags were pulled during this report.

External documentation

E1. Docker: Building best practices
https://docs.docker.com/build/building/best-practices/

E2. Docker Hub: PostgreSQL Official Image
https://hub.docker.com/_/postgres

E3. Docker: Control startup and shutdown order in Compose
https://docs.docker.com/compose/how-tos/startup-order/

E4. Mailpit: Docker images
https://mailpit.axllent.org/docs/install/docker/

E5. NGINX: Unprivileged Dockerfiles
https://github.com/nginx/docker-nginx-unprivileged

External references consulted 20 October 2026. Image versions in this report are those declared in the repository.